HAND-ON LAB 2

Membuat Aplikasi WPF + C# Terhubung SQL Server dengan ADO.NET

Mata Kuliah: .NET Programming with C#
Studi Kasus: Student Registration App
Database: StudentRegistrationDB
Teknologi: C# + WPF + ADO.NET + SQL Server
Target: Menampilkan data Students ke DataGrid

A. Tujuan Praktikum

Setelah menyelesaikan Hands-On Lab 2, mahasiswa mampu:

- Membuat project WPF menggunakan C#.
- Menghubungkan aplikasi WPF dengan SQL Server.
- Menggunakan ADO.NET untuk akses database.
- Membuat Model Student.
- Membuat Model Program.
- Membuat StudentRepository.
- Membuat connection string.
- Mengambil data SQL Server menggunakan SqlConnection.
- Menggunakan SqlCommand dan SqlDataReader.
- Menampilkan data mahasiswa pada DataGrid.

```
WPF UI
   │
   ▼
StudentRepository
   │
   ▼
ADO.NET
   │
   ▼
SQL Server
   │
   ▼
StudentRegistrationDB
   ├── Programs
   └── Students
```

B. Prasyarat

Pastikan Hands-On Lab 1 sudah selesai dan database StudentRegistrationDB telah tersedia.

```
USE StudentRegistrationDB;
GO

SELECT *
FROM Students;
```

Jika data mahasiswa muncul, praktikum dapat dilanjutkan.

C. LAB 01 — Membuat Project WPF

1. Buka Visual Studio.
2. Pilih Create a new project.
3. Cari template WPF.
4. Pilih WPF Application.
5. Pastikan menggunakan C# dan platform Windows.
6. Klik Next.

D. LAB 02 — Memberikan Nama Project

Gunakan nama project:

```
StudentRegistration
```

Pilih framework .NET yang digunakan di laboratorium, misalnya .NET 8, kemudian klik Create.

E. Struktur Awal Project

```
StudentRegistration
├── App.xaml
├── App.xaml.cs
├── MainWindow.xaml
├── MainWindow.xaml.cs
└── StudentRegistration.csproj
```

Struktur target setelah praktikum:

```
StudentRegistration
├── Models
│   ├── Student.cs
│   └── Program.cs
├── Repositories
│   └── StudentRepository.cs
├── MainWindow.xaml
├── MainWindow.xaml.cs
├── App.xaml
├── App.xaml.cs
└── StudentRegistration.csproj
```

F. LAB 03 — Membuat Folder Models

7. Klik kanan project StudentRegistration.
8. Pilih Add → New Folder.
9. Beri nama Models.

G. LAB 04 — Membuat Student.cs

10. Klik kanan folder Models.
11. Pilih Add → Class.

12. Beri nama Student.cs.

```
namespace StudentRegistration.Models;

public class Student
{
    public int StudentId { get; set; }

    public string NIM { get; set; } = string.Empty;

    public string Name { get; set; } = string.Empty;

    public int ProgramId { get; set; }

    public string ProgramName { get; set; } = string.Empty;

    public DateTime? BirthDate { get; set; }

    public string Address { get; set; } = string.Empty;

    public string PhoneNumber { get; set; } = string.Empty;

    public DateTime CreatedAt { get; set; }

    public DateTime? UpdatedAt { get; set; }
}
```

Student.cs merepresentasikan data pada tabel Students. ProgramName ditambahkan untuk menampung hasil JOIN dengan tabel Programs.

H. LAB 05 — Membuat Program.cs

13. Klik kanan folder Models.
14. Pilih Add → Class.
15. Beri nama Program.cs.

```
namespace StudentRegistration.Models;

public class Program
{
    public int ProgramId { get; set; }

    public string ProgramCode { get; set; } = string.Empty;

    public string ProgramName { get; set; } = string.Empty;

    public bool IsActive { get; set; }
}
```

Catatan: Program.cs di sini adalah Model Program Studi, bukan file entry point aplikasi.

I. LAB 06 — Install Microsoft.Data.SqlClient

Buka Tools → NuGet Package Manager → Manage NuGet Packages for Solution. Cari Microsoft.Data.SqlClient kemudian install.

```
using Microsoft.Data.SqlClient;
```

J. LAB 07 — Membuat Folder Repositories

16. Klik kanan project StudentRegistration.
17. Pilih Add → New Folder.
18. Beri nama Repositories.

K. LAB 08 — Membuat StudentRepository.cs

19. Klik kanan folder Repositories.
20. Pilih Add → Class.
21. Beri nama StudentRepository.cs.

L. LAB 09 — Connection String

Untuk SQL Server Express, gunakan connection string berikut. Sesuaikan nama server jika instance SQL Server berbeda.

```
private readonly string _connectionString =
    @"Server=.\SQLEXPRESS;
      Database=StudentRegistrationDB;
      Trusted_Connection=True;
      TrustServerCertificate=True;";
```

M. LAB 10 — Membuat StudentRepository

```
using Microsoft.Data.SqlClient;
using StudentRegistration.Models;

namespace StudentRegistration.Repositories;

public class StudentRepository
{
    private readonly string _connectionString =
        @"Server=.\SQLEXPRESS;
          Database=StudentRegistrationDB;
          Trusted_Connection=True;
          TrustServerCertificate=True;";

    public List<Student> GetAll()
    {
        var students = new List<Student>();

        const string sql = """
            SELECT
                s.StudentId,
                s.NIM,
                s.Name,
                s.ProgramId,
                p.ProgramName,
                s.BirthDate,
                s.Address,
                s.PhoneNumber,
                s.CreatedAt,
                s.UpdatedAt
            FROM Students s
```

```
    INNER JOIN Programs p
        ON s.ProgramId = p.ProgramId
    ORDER BY s.StudentId;
    """;

using var connection =
    new SqlConnection(_connectionString);

using var command =
    new SqlCommand(sql, connection);

connection.Open();

using var reader = command.ExecuteReader();

while (reader.Read())
{
    students.Add(new Student
    {
        StudentId = reader.GetInt32(
            reader.GetOrdinal("StudentId")),

        NIM = reader.GetString(
            reader.GetOrdinal("NIM")),

        Name = reader.GetString(
            reader.GetOrdinal("Name")),

        ProgramId = reader.GetInt32(
            reader.GetOrdinal("ProgramId")),

        ProgramName = reader.GetString(
            reader.GetOrdinal("ProgramName")),

        BirthDate = reader.IsDBNull(
            reader.GetOrdinal("BirthDate"))
            ? null
            : reader.GetDateTime(
                reader.GetOrdinal("BirthDate")),

        Address = reader.IsDBNull(
            reader.GetOrdinal("Address"))
            ? string.Empty
            : reader.GetString(
                reader.GetOrdinal("Address")),

        PhoneNumber = reader.IsDBNull(
            reader.GetOrdinal("PhoneNumber"))
            ? string.Empty
            : reader.GetString(
                reader.GetOrdinal("PhoneNumber")),

        CreatedAt = reader.GetDateTime(
            reader.GetOrdinal("CreatedAt")),

        UpdatedAt = reader.IsDBNull(
            reader.GetOrdinal("UpdatedAt"))
            ? null
            : reader.GetDateTime(
```

```
                        reader.GetOrdinal("UpdatedAt"))
            });
        }

        return students;
    }
}
```

N. Memahami Alur Repository

```
GetAll()
   │
   ▼
SQL Query
   │
   ▼
SqlConnection
   │
   ▼
SqlCommand
   │
   ▼
ExecuteReader()
   │
   ▼
SqlDataReader
   │
   ▼
Student Object
   │
   ▼
List<Student>
```

O. LAB 11 — Memahami ADO.NET

Tiga class penting dalam lab ini:

- SqlConnection — membuka koneksi ke SQL Server.
- SqlCommand — menjalankan perintah SQL.
- SqlDataReader — membaca hasil query.

```
using var connection =
    new SqlConnection(_connectionString);

connection.Open();

using var command =
    new SqlCommand(sql, connection);

using var reader =
    command.ExecuteReader();
```

P. LAB 12 — Mendesain MainWindow

Buka MainWindow.xaml dan ganti isinya dengan:

```
<Window x:Class="StudentRegistration.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Student Registration"
        Height="500"
        Width="900">

    <Grid Margin="20">

        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*"/>
        </Grid.RowDefinitions>

        <StackPanel Orientation="Horizontal"
                    Margin="0,0,0,15">

            <TextBlock Text="Student Registration"
                       FontSize="26"
                       FontWeight="Bold"/>

            <Button Content="Refresh"
                    Width="100"
                    Margin="30,0,0,0"
                    Click="RefreshButton_Click"/>

        </StackPanel>

        <DataGrid x:Name="StudentDataGrid"
                  Grid.Row="1"
                  AutoGenerateColumns="False"
                  IsReadOnly="True"
                  CanUserAddRows="False">

            <DataGrid.Columns>

                <DataGridTextColumn
                    Header="ID"
                    Binding="{Binding StudentId}"
                    Width="60"/>

                <DataGridTextColumn
                    Header="NIM"
                    Binding="{Binding NIM}"
                    Width="100"/>

                <DataGridTextColumn
                    Header="Nama"
                    Binding="{Binding Name}"
                    Width="180"/>

                <DataGridTextColumn
                    Header="Program Studi"
                    Binding="{Binding ProgramName}"
                    Width="180"/>

                <DataGridTextColumn
                    Header="Tanggal Lahir"
                    Binding="{Binding BirthDate, StringFormat=dd/MM/yyyy}"
```

```
                    Width="120"/>

                <DataGridTextColumn
                    Header="No. Telepon"
                    Binding="{Binding PhoneNumber}"
                    Width="140"/>

            </DataGrid.Columns>

        </DataGrid>

    </Grid>

</Window>
```

Q. Tampilan yang Diharapkan

```
┌──────────────────────────────────────────────────────────┐
│ Student Registration                [ Refresh ]          │
├──────────────────────────────────────────────────────────┤
│ ID │ NIM      │ Nama           │ Program Studi │ ...    │
├────┼──────────┼────────────────┼───────────────┼────────┤
│ 1  │ 20231001 │ Ahmad Rizki    │ TI            │ ...    │
│ 2  │ 20231002 │ Siti Aisyah    │ SI            │ ...    │
│ 3  │ 20231003 │ Budi Santoso   │ TI            │ ...    │
│ 4  │ 20231004 │ Citra Lestari  │ Manajemen     │ ...    │
│ 5  │ 20231005 │ Dimas Prayoga  │ Elektro       │ ...    │
└──────────────────────────────────────────────────────────┘
```

R. LAB 13 — Menghubungkan Repository dengan WPF

Buka MainWindow.xaml.cs:

```
using System.Windows;
using StudentRegistration.Repositories;

namespace StudentRegistration;

public partial class MainWindow : Window
{
    private readonly StudentRepository _studentRepository;

    public MainWindow()
    {
        InitializeComponent();

        _studentRepository = new StudentRepository();

        LoadStudents();
    }

    private void LoadStudents()
    {
        try
        {
            var students = _studentRepository.GetAll();

            StudentDataGrid.ItemsSource = students;
```

```
        }
        catch (Exception ex)
        {
            MessageBox.Show(
                ex.Message,
                "Database Error",
                MessageBoxButton.OK,
                MessageBoxImage.Error);
        }
    }

    private void RefreshButton_Click(
        object sender,
        RoutedEventArgs e)
    {
        LoadStudents();
    }
}
```

S. LAB 14 — Menjalankan Aplikasi

22. Tekan F5 atau klik Start.
23. Pastikan aplikasi WPF terbuka.
24. Pastikan data mahasiswa muncul pada DataGrid.
25. Klik Refresh untuk mengambil data kembali dari database.

```
WPF
 │
 ▼
StudentRepository
 │
 ▼
SQL Server
 │
 ▼
StudentRegistrationDB
 │
 ▼
Students
 │
 ▼
List<Student>
 │
 ▼
DataGrid
```

T. Jika DataGrid Kosong

Lakukan troubleshooting secara sistematis.

26. Buka SSMS dan pastikan SELECT * FROM Students menghasilkan data.
27. Pastikan Server pada connection string sesuai dengan instance SQL Server.
28. Pastikan nama database adalah StudentRegistrationDB.
29. Pastikan package Microsoft.Data.SqlClient telah terinstal.
30. Pastikan query JOIN dapat dijalankan langsung di SSMS.

```
SELECT
    s.StudentId,
    s.NIM,
    s.Name,
    s.ProgramId,
    p.ProgramName,
    s.BirthDate,
    s.Address,
    s.PhoneNumber,
    s.CreatedAt,
    s.UpdatedAt
FROM Students s
INNER JOIN Programs p
    ON s.ProgramId = p.ProgramId
ORDER BY s.StudentId;
```

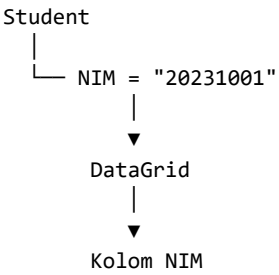
U. LAB 15 — Menguji Query Secara Terpisah

Jika query menghasilkan 5 baris atau sesuai jumlah data di database, maka database dan query sudah benar. Selanjutnya periksa koneksi dan kode C#.

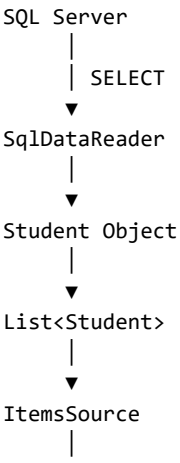
V. LAB 16 — Memahami Data Binding

```
<DataGridTextColumn
    Header="NIM"
    Binding="{Binding NIM}"/>
```

Binding menghubungkan property Student.NIM dengan kolom NIM pada DataGrid.



W. LAB 17 — Dari Database ke DataGrid



```
    ▼
DataGrid
```

```
var students =
    _studentRepository.GetAll();

StudentDataGrid.ItemsSource =
    students;
```

X. LAB 18 — Memahami Tanggung Jawab Class

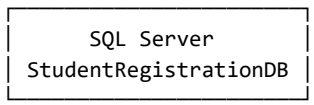
Class/File	Tanggung Jawab
Student.cs	Model data mahasiswa
Program.cs	Model program studi
StudentRepository.cs	Data access ke SQL Server
MainWindow.xaml	User Interface WPF
MainWindow.xaml.cs	Event dan UI logic tahap awal

Y. Struktur Project Setelah Lab 2

```
StudentRegistration
│
├── Models
│   ├── Student.cs
│   └── Program.cs
│
├── Repositories
│   └── StudentRepository.cs
│
├── MainWindow.xaml
├── MainWindow.xaml.cs
│
├── App.xaml
├── App.xaml.cs
│
└── StudentRegistration.csproj
```

Z. Arsitektur yang Sudah Terbentuk

```
┌──────────────────────┐
│      MainWindow      │
│       WPF UI         │
└──────────┬───────────┘
           │
           ▼
┌──────────────────────┐
│  StudentRepository   │
│     Data Access      │
└──────────┬───────────┘
           │
           ▼
┌──────────────────────┐
│       ADO.NET        │
└──────────┬───────────┘
           │
           ▼
```



AA. LAB 19 — Mini Challenge

Challenge 1 — Tambahkan Address ke DataGrid:

```
<DataGridTextColumn
    Header="Alamat"
    Binding="{Binding Address}"
    Width="250"/>
```

Challenge 2 — Tambahkan CreatedAt:

```
<DataGridTextColumn
    Header="Dibuat"
    Binding="{Binding CreatedAt}"
    Width="150"/>
```

Challenge 3 — Tambahkan Total Mahasiswa. Gunakan query:

```
SELECT COUNT(*)
FROM Students;
```

AB. LAB 20 — Assignment

Buat aplikasi dengan minimal fitur:

- WPF Project.
- Student.cs.
- Program.cs.
- StudentRepository.cs.
- Connection String.
- SqlConnection.
- SqlCommand.
- SqlDataReader.
- List<Student>.
- DataGrid.
- Tombol Refresh.
- Data SQL Server berhasil ditampilkan.

AC. Checkpoint Hands-On Lab 2

Database:

☐ StudentRegistrationDB

☐ Programs

☐ Students

C#:

☐ Student.cs

☐ Program.cs

☐ StudentRepository.cs

ADO.NET:

☐ Microsoft.Data.SqlClient

☐ SqlConnection

☐ SqlCommand

☐ SqlDataReader

WPF:

☐ MainWindow.xaml

☐ DataGrid

☐ Refresh Button

☐ Data berhasil ditampilkan

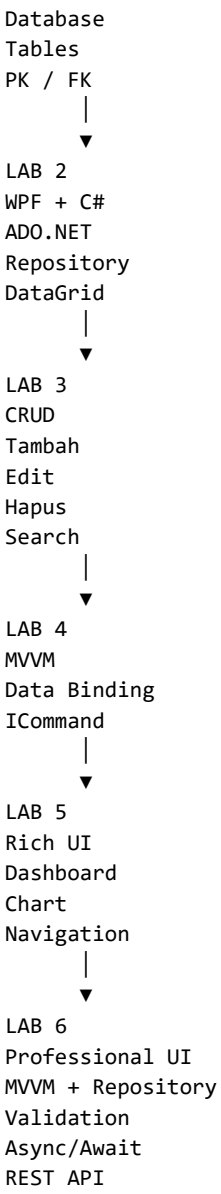
AD. Konsep yang Harus Dipahami Mahasiswa

Pada akhir Lab 2, mahasiswa harus dapat menjelaskan bagaimana data dari SQL Server sampai ke DataGrid WPF:

```
1. User menjalankan aplikasi WPF
            ↓
2. MainWindow membuat StudentRepository
            ↓
3. Repository membuat SqlConnection
            ↓
4. SqlCommand menjalankan SELECT
            ↓
5. SqlDataReader membaca hasil SQL Server
            ↓
6. DataReader dipetakan menjadi object Student
            ↓
7. Object Student dimasukkan ke List<Student>
            ↓
8. List<Student> menjadi ItemsSource
            ↓
9. DataGrid menampilkan data
```

AE. Roadmap Pembelajaran

```
LAB 1
SQL Server
```



Dengan urutan ini, mahasiswa belajar dari database → koneksi → data access → UI, sebelum masuk ke MVVM dan arsitektur aplikasi profesional.